A Computer Vision Project Report

MIDTERM PROJECT REPORT

Submitted as a requirement for the course
Computer Vision

Group Project Specialization
Object Sorting

Prepared by:

Alba, John Raymond S.
Bacsain, Jave A.
Bagaporo, John Louie B.
Nonato, Francis Gabriel F.
Palacios, Harvey Lloyd V.
Prestado, Marc Justin N.

BS Computer Science – 4A

Submitted to:

Ms. Elena Clarexe R. Gulane
Course Instructor

1. INTRODUCTION

1.1 Project Overview

This project develops the computer vision component of a vision-based autonomous 4WD robot assigned to the Object Sorting specialization. Currently, the proposed system asks one question: can the system itself see and recognize objects in front of the camera? Given an image or a live camera frame of a waste item, the system must identify which type of material it is and show that result visibly on screen. The robot is not required to move at this stage. In the Final stage, the recognized class will drive the robot's decision and motor control so that it can physically sort objects.

The Computer Vision problem addressed in this project is a multi-class image classification. We train a convolutional neural network (CNN) on the RealWaste dataset (Single et al., 2023), sourced from UC Irvine Machine Repository, a public collection of 4,752 photographs of waste items taken at the point of reception at a landfill in Wollongong, Australia. Unlike studio-quality datasets, these images show objects in cluttered, uneven, real-life conditions, which makes them a more realistic test of whether a classifier can handle the visual variation a sorting robot would face.

1.2 Problem Description

Sorting waste by material is a repetitive task that is slow, inconsistent when done by hand, and unpleasant or unsafe for the people doing it. Mistakes contaminate recyclable material and reduce its value. An automated sorter first needs a reliable way to tell what an object is from a camera image alone.

This is difficult for a computer vision system for several reasons. Items of the same class vary widely in shape, size, color and condition (a crushed bottle and an intact bottle are both plastic). Different classes can look alike (paper and cardboard share color and texture; clear glass and clear plastic reflect light similarly). Items may also be partly hidden, deformed, dirty, or photographed under changing lighting and backgrounds.

The current problem statement for this project would be: given a single RGB image of one waste item, the system must assign it to exactly one of the selected classes, report a confidence score, and generalize to images it has not seen during training.

1.3 Project Objectives

General objective:

To design, train and evaluate a computer vision system that recognizes selected waste object classes from images and live camera input, forming the vision foundation of an object-sorting robot.

Specific objectives:

1. To prepare a dataset of five waste classes with at least 100 images per class, divided into separate training, validation and testing sets.

2. To apply image preprocessing (resizing, color conversion, normalization) and data augmentation to prepare images for the model.
3. To train a CNN classifier using transfer learning and tune it using the validation set.
4. To evaluate the model on the held-out test set using accuracy, precision, recall, F1-score and a confusion matrix, with a target of at least 90% overall test accuracy.
5. To demonstrate the complete pipeline (Camera/Image → Preprocessing → Model → Recognition → Visual Output) on images chosen by the instructor and on live camera input, displaying the predicted class label and confidence score.
6. To check generalization on an external set of openly licensed public images under different lighting, backgrounds and viewing angles than RealWaste's own training photos.

1.4 Scope and Limitations

The current project progress covers the Computer Vision stage of Object Sorting for the Midterm. It does not cover robot motion or sorting mechanics.

- **Classes/categories.** Five material classes from the RealWaste dataset: Cardboard, Glass, Metal, Paper and Plastic. The dataset contains nine classes; the other four (Food Organics, Miscellaneous Trash, Textile Trash and Vegetation) are excluded to remain within the Midterm limit of three to five classes. The subset contains 3,092 images, well above the minimum of 100 per class.

Class	Total images	Training (70%)	Validation (15%)	Testing (15%)
Cardboard	461	323	69	69
Glass	420	294	63	63
Metal	790	553	118	119
Paper	500	350	75	75
Plastic	921	644	139	138
Total	**3,092**	**2,164**	**464**	**464**

Table 1. Class distribution and 70/15/15 stratified split of the RealWaste subset (as executed; see Section 3.5).

The subset is uneven: Plastic contributes 921 images and Glass 420. The stratified split keeps each class in the same proportion inside the training, validation and testing sets, so no split is made easier or harder than the others by the imbalance.

- **Objects/targets**. A single waste item per image, belonging to one of the five classes above, presented reasonably close to the camera.
- **Visual tasks.** Image classification: predicting the class label and a confidence score for the whole image.

- **Operating scope**. Still images from the dataset, instructor-provided images, and live camera input of items held or placed in front of the camera in an indoor setting with reasonable lighting. Processing runs on a laptop/PC.

Relevant limitations.

- The training images come from one landfill facility and one camera setup, so accuracy may drop on images with very different backgrounds, lighting or camera quality. This is confirmed by the external cross-dataset test in Section 6.5: accuracy drops from 87.93% on RealWaste's own held-out test split to 56.00% on 500 everyday TrashNet photos.
- The system can only output one of the five trained classes. An object outside these classes (for example food waste or fabric) will still be forced into the closest of the five.
- The dataset is imbalanced (Plastic has 921 images and Glass 420), which may bias predictions toward larger classes unless handled during training.
- Visually similar classes (Paper and Cardboard, Glass and Plastic) are expected to be the main source of errors.

1.5 Group Project Specialization

The group's assigned specialization is **Object Sorting**. The required Computer Vision task is to recognize the selected object classes so that a robot can later sort them. The system must correctly identify the selected classes from images and live camera input and show a visible result (class label and confidence score). The recognition result will be used for decision logic and motor control to move and sort objects autonomously.

2. SYSTEM DESIGN AND ARCHITECTURE

2.1 Overall System Architecture

Figure 1 shows the system architecture of our project. Part A is the pipeline run at demonstration time: an input image or camera frame is preprocessed, passed to the trained CNN, converted to a class prediction with a confidence score, and displayed on the frame. Part B is the offline pipeline that produces the trained model: the dataset subset is split, preprocessed and augmented, used to train the network, and finally evaluated on the test set.

The live demonstration extends Part A with remote camera input: a groupmate's phone opens the demo page in a browser and streams camera frames, or sends a single captured photo as a fallback. These frames reach the processing computer through a secure HTTPS relay hosted on a cloud server, needed because the processing computer sits on a private network without a public address of its own. The relay only forwards encrypted traffic; all image decoding, preprocessing, and classification still happen on the processing computer, using the same Part-A pipeline as local images. Results (label, confidence, and the top-3 class probabilities) are sent back to the originating phone and, together with every other connected phone, to a shared Presenter view so the pipeline can be shown live to an audience (Section 5.2, 5.3). Figure 1 is repeated in Appendix E.

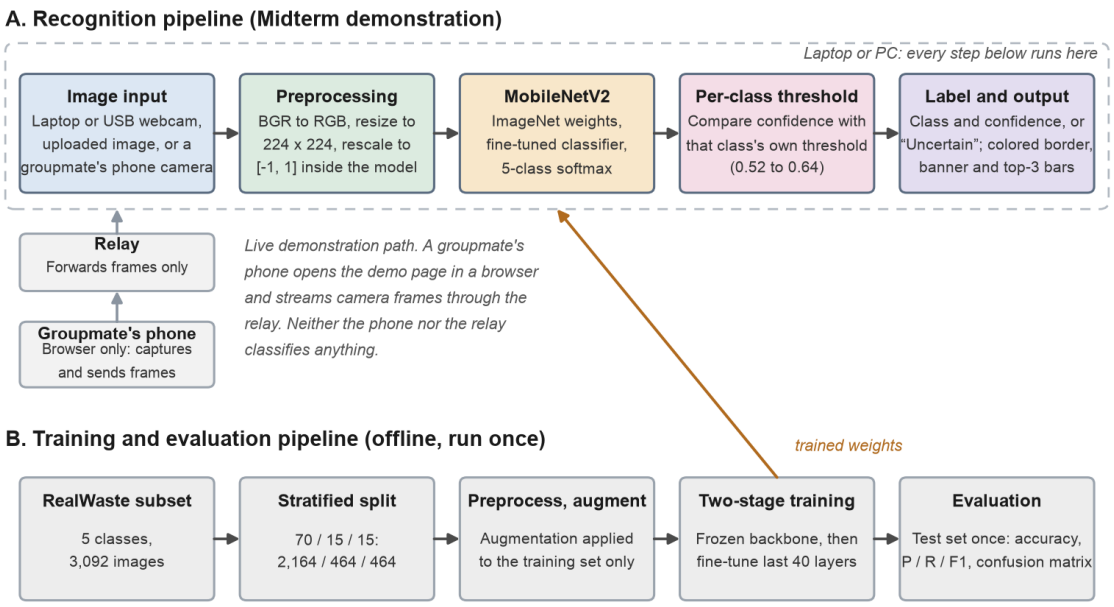


Figure 1. System Architecture for Object Sorting

The dashed box marks the processing computer: every step of Part A happens there, including the per-class threshold that decides between a class label and “Uncertain”. The phone and the relay below it only move the picture.

2.2 Computer Vision Workflow

The Computer Vision system performs the following sequence of operations:

1. **Dataset preparation.** Match the five class names to their image folders inside the extracted RealWaste zip (524 × 524 pixel color images); images are filtered by file extension only (no explicit corruption check beyond decoding during preprocessing).
2. **Dataset splitting.** Divide each class into training, validation and testing sets using a stratified 70/15/15 split. The split is done before augmentation so that no test or validation image (or a modified copy) is seen during training.
3. **Preprocessing.** Resize every image to 224 × 224 pixels, ensure RGB channel order, and scale pixel values to the range expected by the network.
4. **Data augmentation (training set only).** Apply random flips, rotations, zoom, brightness and contrast changes so the model learns to handle different lighting and viewing angles.
5. **Model construction.** Use MobileNetV2 pre-trained on ImageNet as a feature extractor, followed by global average pooling, dropout, and a dense softmax layer with five outputs.
6. **Training.** Train the new classification layers with the Adam optimizer and sparse categorical cross-entropy loss (integer class labels), apply class weights to reduce the effect of class imbalance, then fine-tune the last 40 layers of the backbone at a lower learning rate. EarlyStopping (patience 4, restore_best_weights=True) and ReduceLROnPlateau watch validation loss throughout; see Section 4.4.

7. **Evaluation.** Run the final model once on the untouched test set and compute accuracy, per-class precision, recall and F1-score, and a confusion matrix. An external cross-dataset test of openly licensed public images is prepared separately (Section 6.5).
8. **Recognition and visual output.** For a new image or live camera frame, apply the same preprocessing, run the model, take the class with the highest softmax probability, and draw the class label and confidence score on the image. Predictions below a confidence threshold are marked as uncertain. Frames streamed from a phone browser through the relay in Section 2.1 go through this same step, so remote and local images are classified identically.

2.3 Hardware Components

Listed below are the proposed hardware components relevant to the Computer Vision prototype.

Component	Specification / Model	Purpose
Camera	Laptop/PC built-in or USB webcam (local fallback; used for development and testing)	Captures live frames of the object to be recognized
Camera clients (remote)	Groupmates' own smartphones (browser only; no app installed)	Stream or capture frames for the Phone camera tab, over the relay in Section 2.1/2.4
Controller	Laptop/PC (CPU)	Runs preprocessing and model inference for every connected camera, local or remote
Power Source	Laptop power / battery pack	Powers the processing unit
Others	Sample waste items (bottle, can, paper, cardboard, glass); plain background; adequate lighting	Live demonstration

Table 2. Hardware used by the Computer Vision prototype.

The prototype runs on ordinary equipment. One laptop or PC does the preprocessing and inference for every connected camera, and the phones listed as remote camera clients only capture and send pictures.

2.4 Software and Development Tools

The system is implemented in Python using open-source libraries. All external datasets, libraries and pre-trained models are acknowledged in the References.

Software / Tool	Version	Purpose
Python	3.11.9	Main programming language for data preparation, training, evaluation and the demonstration program

Software / Tool	Version	Purpose
TensorFlow	2.21.0	Deep learning framework. Loads and resizes images (tf.data, tf.image), trains the CNN and runs predictions
Keras (part of TensorFlow)	3.15.1	High-level API used to build the model (augmentation layers, pooling, dropout, softmax), compile it, train it with EarlyStopping/ReduceLROnPlateau, and save it as .keras
MobileNetV2 (pre-trained on ImageNet)	Keras Applications, 224×224, include_top=False	Pre-trained CNN backbone used for transfer learning and fine-tuning
OpenCV (opencv-python)	4.14.0 (as printed by the notebook at run time)	Reads images, converts BGR to RGB, resizes for the demo, and draws the label, confidence and box on the output image. Also supports webcam frames via cv2.VideoCapture
Gradio	6.30.0	Serves the local demo web app (upload, live camera, phone camera, and presenter tabs) and handles image upload and streaming in the browser
Reverse SSH tunnel + HTTPS reverse proxy (Caddy, cloud server)	n/a	Relays encrypted traffic from the public internet to the processing computer, which has no public address of its own, so phones can reach the demo over HTTPS (required for browser camera access); only forwards traffic, does not run any classification
NumPy	2.4.6	Array math: labels, probabilities, argmax, normalizing the confusion matrix
pandas	2.2.3	Tables for class counts, the train/val/test split and the classification report. Exports split.csv
Matplotlib	3.11.1	Plots the class distribution, sample images, augmentation preview, training curves, confusion matrix and prediction results
scikit-learn	1.9.1	Stratified train/val/test split (train_test_split), balanced class weights, and evaluation (accuracy, classification report, confusion matrix)
RealWaste dataset	UCI ML Repository, 2023 (DOI 10.24432/C5SS4G, CC BY 4.0)	Image dataset; 5 classes are used (Cardboard, Glass, Metal, Paper, Plastic), 3,092 images
Python standard library (os, pathlib, zipfile, shutil, json, random, time, glob)	Comes with Python 3.11.9	File handling, extracting the dataset zip, random seeds, timing, saving results.json and class_names.json

Table 3. Software and development tools, with the versions used.

These are the versions used for the training run and the demonstration reported here. One row is not software: the RealWaste dataset is listed with its DOI and license because the whole pipeline depends on it.

3. DATASET

3.1 Dataset Description

The Computer Vision system is developed and evaluated on RealWaste (Single et al., 2023), a public dataset of 4,752 photographs of waste items taken at the point of reception at a landfill in Wollongong, Australia. The dataset provides 524 × 524 pixel RGB color images organized into 9 material classes. Because the photographs were taken under real landfill conditions rather than a controlled studio setup, items appear cluttered, at varied distances, and under uneven lighting, which the group selected deliberately since a sorting robot will face the same kind of visual variation.

3.2 Target Classes / Categories

The group uses 5 of the 9 RealWaste classes, which satisfies the Midterm limit of 3–5 classes and exceeds the Specialization 8 (Object Sorting) minimum of 4 classes.

Class / Category	Description	Number of Samples
Cardboard	Corrugated/flat cardboard pieces and boxes	461
Glass	Glass bottles and jars (clear and colored)	420
Metal	Metal cans and metal containers	790
Paper	Paper sheets, cups, and paper packaging	500
Plastic	Plastic bottles, containers, and packaging	921
Total		3,092

Table 4. Target classes and sample counts for the selected RealWaste subset.

The five classes are the RealWaste categories a sorting robot would have to separate, and every one of them is far above the Midterm minimum of 100 images: the smallest, Glass, has 420.

3.3 Dataset Collection and Source

RealWaste is sourced from the UC Irvine Machine Learning Repository (https://doi.org/10.24432/C5SS4G), released under a CC BY 4.0 license by Single, Iranmanesh, and Raad (2023). The notebook (src/training/Midterm_Object_Sorting_RealWaste.ipynb)

downloads the dataset zip directly from the UCI static URL at run time, so no copy of the images is stored in this repository; see dataset/README.md for the exact source link and citation.

3.4 Dataset Distribution

The selected 5-class subset is imbalanced: Plastic (921 images) has about 2.2× as many images as Glass (420), the smallest class (see Table 4 in Section 3.2 for the full per-class counts). This imbalance is handled during training with class weights computed by sklearn's compute_class_weight('balanced', ...); see Section 4.4.

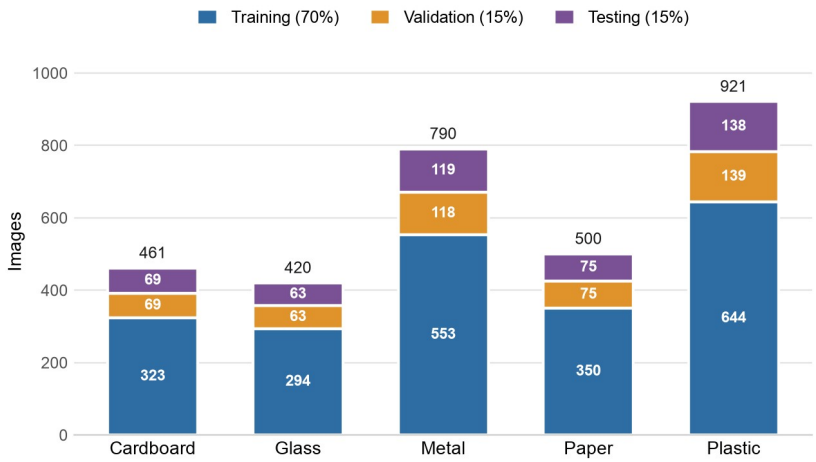


Figure 2. Images per class in the selected RealWaste subset, split into training, validation and testing sets.

Plastic is the largest class with 921 images and Glass the smallest with 420, about 2.2 times fewer. The stratified split keeps that same proportion inside each of the three sets, so the imbalance is spread evenly across training, validation and testing rather than concentrated in one of them.



Figure 3. Sample images from each of the 5 RealWaste classes used in this project.

The samples show why the task is not trivial. Items appear crushed, torn, dirty or partly transparent, and several classes share the same muted colors and textures, which is the visual similarity described in Section 1.2.

3.5 Training, Validation, and Testing Sets

The 3,092 selected images are split 70% training / 15% validation / 15% testing using a stratified split (sklearn train_test_split, seed 42) so each split preserves the same per-class proportions. The split is performed on file paths before any augmentation, and the notebook asserts that no path appears in more than one split. See Table 1 (Section 1.4) for the full per-class breakdown: 2,164 training / 464 validation / 464 testing images.

- **Training set (2,164 images).** Used to update the model's weights during both training stages.
- **Validation set (464 images).** Used only to monitor validation loss for EarlyStopping / ReduceLROnPlateau; never used to update weights directly.
- **Testing set (464 images).** Held out completely and evaluated exactly once, after training is finished, to report the accuracy and other metrics in Section 6.

3.6 Dataset Preparation and Preprocessing

1. **Decoding.** Images are decoded as RGB with tf.io.decode_image.
2. **Resizing.** Every image is resized to 224×224 with antialiasing (tf.image.resize).
3. **Rescaling.** Pixel values are kept as float32 in the notebook's data pipeline; the rescaling to the [-1, 1] range MobileNetV2 expects is performed inside the model itself (a Rescaling(1/127.5, offset=-1) layer), so the same raw-pixel input works for both training and the recognize() demo function.
4. **Augmentation (training set only).** Random horizontal and vertical flip, rotation (±15%), zoom (±15%), brightness (±20%) and contrast (±20%) are applied to the training set only, so the model sees more lighting and viewing-angle variation without changing the validation and testing data used for evaluation.

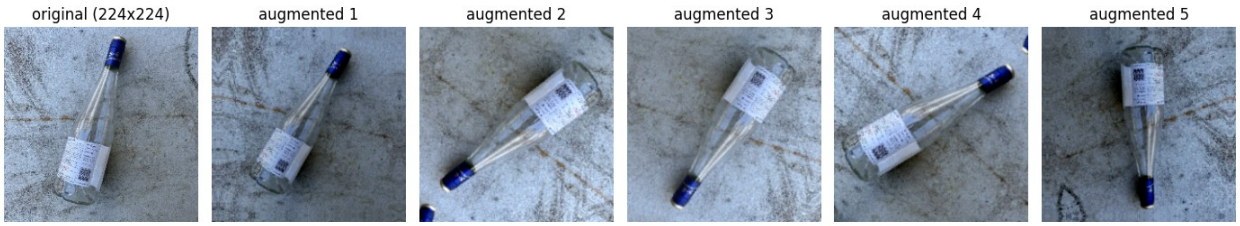


Figure 4. One training image (left) and five randomly augmented versions used only during training.

Each augmented copy is the same training image under a different flip, rotation, zoom, brightness or contrast, so the model sees more lighting and viewing-angle variation than the dataset itself contains. The validation and testing images are left untouched.

4. COMPUTER VISION METHODOLOGY

4.1 Computer Vision Approach

The group uses supervised image classification via transfer learning rather than a classical (non-learning) CV algorithm or training a CNN from scratch. With only 3,092 images across 5 classes, training a deep CNN from scratch risks overfitting; starting from MobileNetV2 weights pre-trained on ImageNet gives the model general visual features (edges, textures, shapes) that are then adapted to the waste-sorting task. MobileNetV2 is also a lightweight architecture, which keeps inference fast enough for the live recognize() demo.

4.2 Image Processing / Preprocessing

See Section 3.6 for the full preprocessing pipeline (resize to 224×224, RGB decoding, in-model rescaling to [-1, 1], training-only augmentation). Training and evaluation resize with tf.image.resize(antialias=True) through tf.data; the recognize() demo function resizes the same way conceptually but with OpenCV (cv2.cvtColor + cv2.resize, INTER_AREA) instead, since it works on one image or camera frame at a time rather than a tf.data pipeline. Both paths decode RGB, resize to 224×224, and rely on the same in-model rescaling, but the exact resize implementation differs between the two, which is a source of small numeric difference between a tf.data evaluation and a recognize() call on the same image.

4.3 Model / Algorithm

The model (build_model() in the notebook) is a MobileNetV2 backbone (keras.applications.MobileNetV2, ImageNet weights, include_top=False, 224×224×3 input) used as a feature extractor, followed by:

- **GlobalAveragePooling2D:** collapses the backbone's feature maps to one vector per image.
- **Dropout(0.3):** randomly zeroes 30% of the pooled features during training, to reduce overfitting.
- **Dense(5, activation='softmax'):** the classification head producing a probability for each of the 5 classes.

4.4 Model Training / Development

Training runs in two stages, both using the Adam optimizer and sparse categorical cross-entropy loss, with class weights applied in every epoch of both stages:

Setting	Backbone	Epoch budget	Learning rate
Stage 1 (head)	frozen	up to 12 (EarlyStopping, patience 4)	1e-3
Stage 2 (fine-tune)	last 40 layers unfrozen (BatchNorm frozen)	up to 10 (EarlyStopping, patience 4)	1e-5

Table 5. Training hyperparameters for the reported run.

EarlyStopping (patience 4, restore_best_weights=True) and ReduceLROnPlateau (factor 0.5, patience 2) watch validation loss in both stages, so the kept weights are the best-validation-loss checkpoint, not necessarily the last epoch.

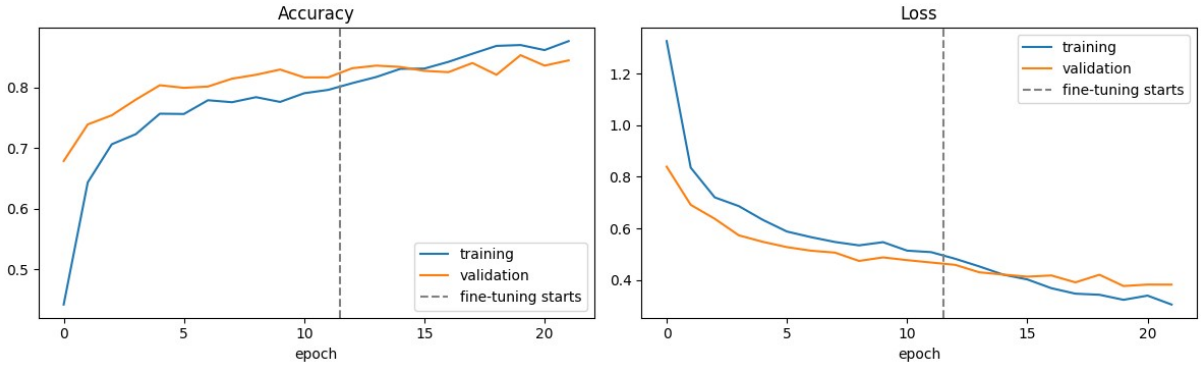


Figure 5. Training and validation accuracy (left) and loss (right) per epoch for both training stages, from the group's Colab run of the same pipeline and settings (that run reached 88.15% test accuracy; the final reported model, 87.93%, was trained identically).

Validation accuracy leads training accuracy through the first stage, where dropout and the augmentation layers are active on the training batches only. After the first 12 epochs, where fine-tuning starts (the dashed line), the training curve overtakes it: validation accuracy peaks late in the run at about 0.85 while validation loss flattens near 0.38 and training loss keeps falling, the mild overfitting that EarlyStopping with restore_best_weights is there to catch.

4.5 Model Parameters and Configuration

Parameter	Value	Notes
Classes	5 (Cardboard, Glass, Metal, Paper, Plastic)	fixed order used for argmax to label
Input size	224 × 224 × 3	matches MobileNetV2's expected input
Batch size	32	
Random seed	42	used for the split and all shuffling
Pre-trained weights	ImageNet	keras.applications.MobileNetV2

Parameter	Value	Notes
Fine-tuned layers	last 40 layers of the backbone	BatchNorm layers kept frozen
Class weighting	sklearn compute_class_weight('balanced')	applied in both training stages
Confidence threshold	per-class (0.52-0.64); global fallback 0.59	selected on the validation set, not hand-picked; see Table 7 and Section 4.6

Table 6. Model parameters and configuration used by the final implementation.

The confidence threshold in the last row is not a single number: each class has its own, listed below.

Class	Threshold (validation)
Cardboard	0.64
Glass	0.58
Metal	0.52
Paper	0.55
Plastic	0.60

Table 7. Per-class confidence thresholds, selected on the validation set (Section 4.6).

A reject option, allowing the classifier to output “Uncertain” rather than forcing a prediction, was included because low-confidence predictions are more likely to be wrong (Hendrycks & Gimpel, 2017), and because a formal basis exists for trading prediction coverage against accuracy in exactly this way (Chow, 1970). CLAMP (Thakkar et al., 2025), a multimodal robot-perception system, independently diverted low-confidence predictions to a separate “I'm not sure” bin in a recycling-sorting demonstration, supporting a reject option as established applied practice in waste sorting and not merely a theoretical construct.

Per-class confidence thresholds were then selected on held-out validation data using a risk-coverage criterion: for each predicted class, the smallest threshold that achieved at least 90% precision on that class's validation predictions was chosen, following the risk-coverage framework for validation-based threshold selection introduced by Geifman and El-Yaniv (2017). Because Chow's (1970) original rule assumed a single global threshold under symmetric misclassification costs, and a single global threshold can be suboptimal when per-class error rates differ or posterior estimates are imperfect, this project adopted class-dependent thresholds, extending Chow's rule in the manner proposed by Fumera, Roli, and Giacinto (2000) for multiple, class-related reject thresholds, with a single global threshold used as a fallback for any class with fewer than 20 validation predictions to avoid an unstable precision estimate from a small sample.

Calibration mattered for interpreting these thresholds because modern neural networks are often overconfident, meaning a raw softmax value does not necessarily reflect the true probability of

correctness (Guo et al., 2017). Calibration was therefore assessed using Expected Calibration Error (ECE), a metric that compares a model's predicted confidence to its empirical accuracy across binned predictions (Naeini, Cooper, & Hauskrecht, 2015). Because the thresholds used in this project were class-specific rather than global, calibration was assessed per class using Static Calibration Error (SCE), a classwise extension of ECE (Nixon et al., 2019); Kull et al. (2019) report a similarly named classwise-ECE metric, though its exact formula could not be fully verified and this comparison is noted as lower confidence. The resulting thresholds were Cardboard 0.64, Glass 0.58, Metal 0.52, Paper 0.55, Plastic 0.60, yielding a test coverage of 88.58%, a selective accuracy of 93.19% on the retained predictions, and an SCE of 2.47%. These values were determined empirically from validation and test data rather than fixed in advance. Softmax confidence values were therefore not treated as true probabilities, since they are often overconfident in the absence of calibration (Guo et al., 2017). The validation sweep and the selection rule behind these thresholds are reproduced in Appendix A.

4.6 Detection / Recognition Process

Camera Frame / Image, then BGR to RGB conversion, then resize to 224×224 (cv2.INTER_AREA), then model.predict(), then take the class with the highest softmax probability; if that probability is ≥ the predicted class's own confidence threshold (Table 7), show the class label and confidence, otherwise show “Uncertain”, and draw the result as a colored border and text banner on the output image (Section 5.3).

5. SYSTEM IMPLEMENTATION

5.1 Computer Vision Implementation

The demonstration pipeline is implemented as a single recognize(source, show=True) function in the notebook (Step 7). It accepts either an image file path or a BGR NumPy array (e.g. a camera frame), applies the Section 4.6 pipeline, and returns the predicted label, its confidence, and a copy of the input image annotated with the result. This keeps the training pipeline (tf.data, Section 3–4) and the single-image demo pipeline (OpenCV) using the same trained model and the same class order, so predictions from either path are directly comparable (see Section 4.2 for the one implementation difference between them). The full implementation, including this notebook, the live demo, and the trained model, is published in the project's GitHub repository (https://github.com/J4ve/cv-object-sorting). Setup and run instructions for the live, phone-accessible version of this demo (used when presenting over Google Meet) are in docs/remote-demo.md in the repository. The notebook can also be opened directly as a Google Colab copy. Both links are repeated in Appendix F.

5.2 Input and Camera Processing

Three input paths are supported: (1) images already in the test split, (2) a file uploaded through the Colab file picker, and (3) a live snapshot captured from the browser's webcam via a small JavaScript helper (capture_from_webcam()) that returns a single JPEG frame. Outside Colab,

the same recognize() function accepts a frame read with cv2.VideoCapture(0), so the pipeline is not tied to the Colab-only webcam helper.

The live demo (src/vision/app.py) extends this with four tabs covering every input path used in practice: Upload / batch (one or more image files, plus one-click RealWaste sample images), Live camera (the processing computer's own webcam, one snapshot per click), Phone camera (a connected phone's rear camera streamed live at about 5 frames per second over the relay in Section 2.1, with a Send photo fallback that opens the phone's native camera app when live streaming is not available), and Presenter (no camera input of its own; it only displays results from connected phones). Phone streaming defaults to the rear camera, with an in-page Switch camera control and automatic correction if a phone's browser ignores that preference.

5.3 Visual Output

Each recognized image is annotated with: a colored border (green if confident, orange if below the predicted class's own threshold, Table 7) drawn with cv2.rectangle, and a text banner in the top-left corner showing the predicted class label and confidence percentage, drawn with cv2.putText. The full per-class softmax probabilities are also printed below the image.

The live demo additionally shows the predicted label and confidence in large text and a bar for each of the top-3 class probabilities. When a prediction falls below the predicted class's own threshold (Table 7), it shows “Uncertain” together with the best-guess class, its confidence, and the threshold it fell short of (for example, “best guess: Plastic 55.0%, below Plastic's 60%”). The Presenter tab shows this same label/confidence/bars display for a single selected phone, or a grid of every connected phone at once, so an audience can watch multiple live classifications side by side.

6. TESTING AND EVALUATION

6.1 Computer Vision Testing

The trained model is evaluated exactly once on the 464-image testing split (Section 3.5), which was never used for training or validation. model.predict() is run over the whole test set in a single call (test_ds, not shuffled, so predictions line up index-for-index with the true labels) and the argmax of each softmax output is compared against the true class.

6.2 Evaluation Metrics

Overall accuracy, per-class precision/recall/F1-score, macro F1, and a confusion matrix (counts and row-normalized) are computed. These are the metrics the Midterm requirements call for a classification task (accuracy metrics and a confusion matrix).

6.3 Training, Validation, and Testing Results

Final test-set results:

- **Test accuracy: 87.93%** (408 of 464 test images correctly classified).

- **Macro F1: 0.878**

Class	Precision	Recall	F1-score
Cardboard	0.8806	0.8551	0.8676
Glass	0.8485	0.8889	0.8682
Metal	0.9052	0.8824	0.8936
Paper	0.8353	0.9467	0.8875
Plastic	0.9000	0.8478	0.8731

Table 8. Per-class precision, recall, and F1-score on the test set.

This falls short of the 90% test-accuracy objective stated in Section 1.3 by 2.07 percentage points; the gap and likely causes are discussed in Section 7.

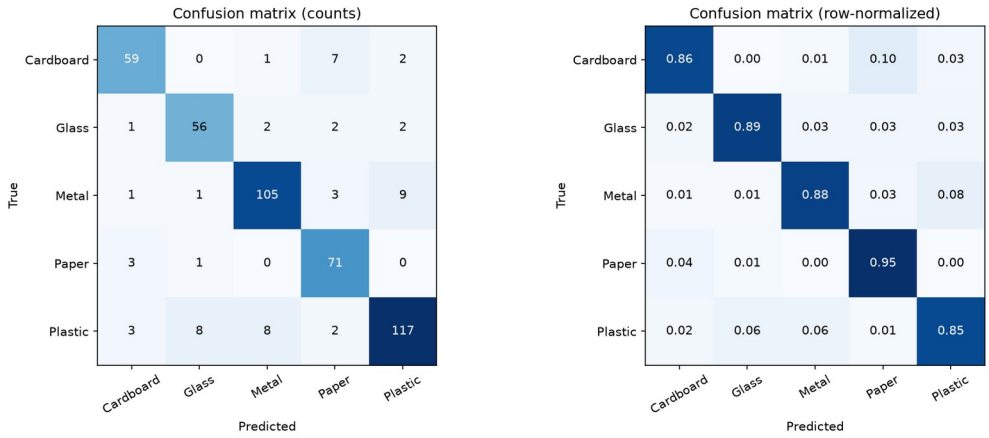


Figure 6. Confusion matrix on the 464-image test set (counts, left; row-normalized, right).

Most common mistakes (true class misread as predicted class, with count): Metal misread as Plastic (9), Plastic misread as Glass (8), Plastic misread as Metal (8), Cardboard misread as Paper (7), Paper misread as Cardboard (3). This confusion matrix is repeated in Appendix B.

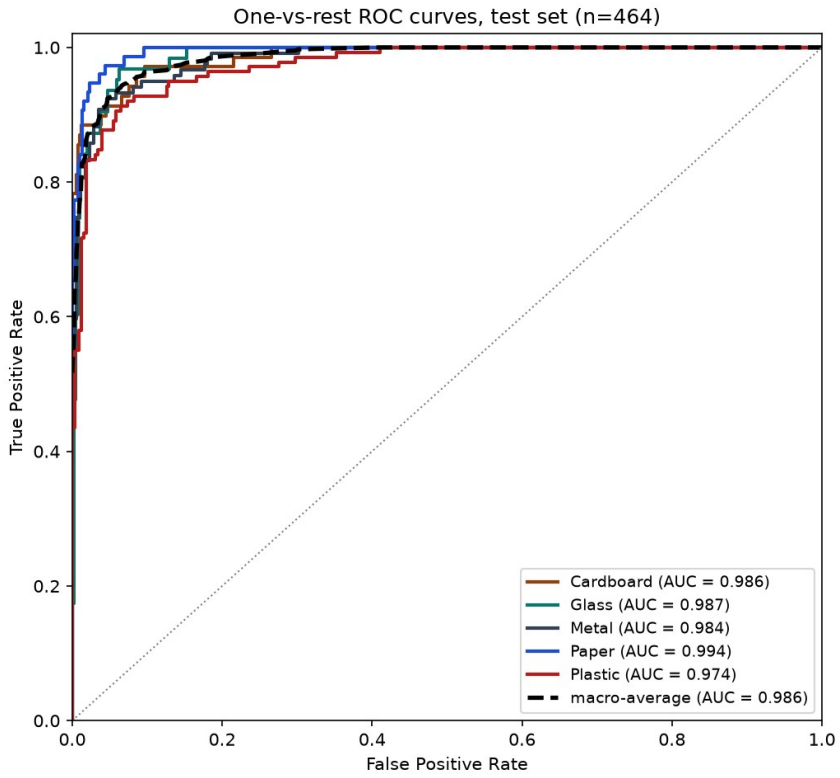


Figure 7. One-vs-rest ROC curves per class on the test set; macro-average AUC = 0.986.

ROC AUC (Figure 7) summarizes how well confidence ranks true positives above false positives per class, independent of any threshold: macro-average AUC 0.986, ranging from 0.974 (Plastic) to 0.994 (Paper).

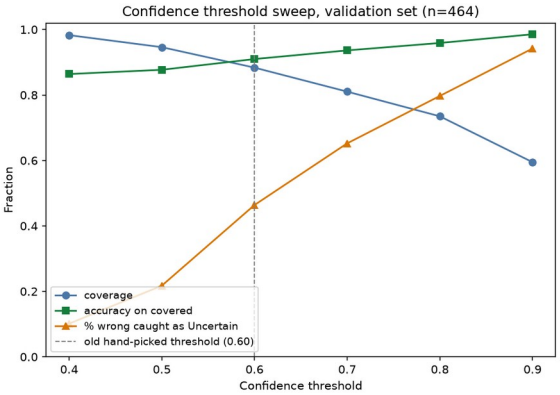


Figure 8. Confidence threshold sweep on the validation set: coverage, accuracy on covered predictions, and the share of wrong predictions caught as Uncertain, at six thresholds.

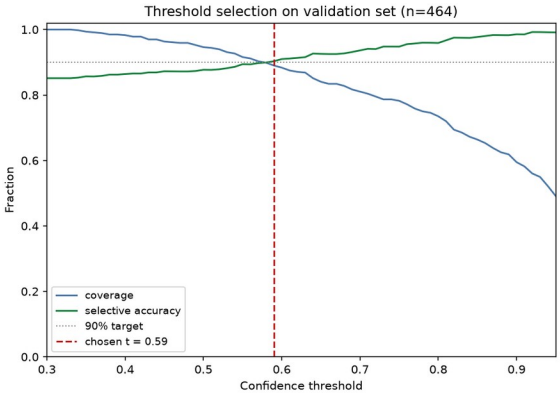


Figure 9. Threshold selection on the validation set: smallest t reaching 90% selective accuracy is 0.59 (dashed line), used as the global fallback.

Figure 8 shows the validation data behind the threshold choice in Section 4.5/4.6: raising the threshold trades coverage for accuracy on the predictions that are kept.

The global fallback threshold (0.59) was selected this way (Figure 9); per-class thresholds (Table 7) were selected the same way per predicted class (Section 4.6).

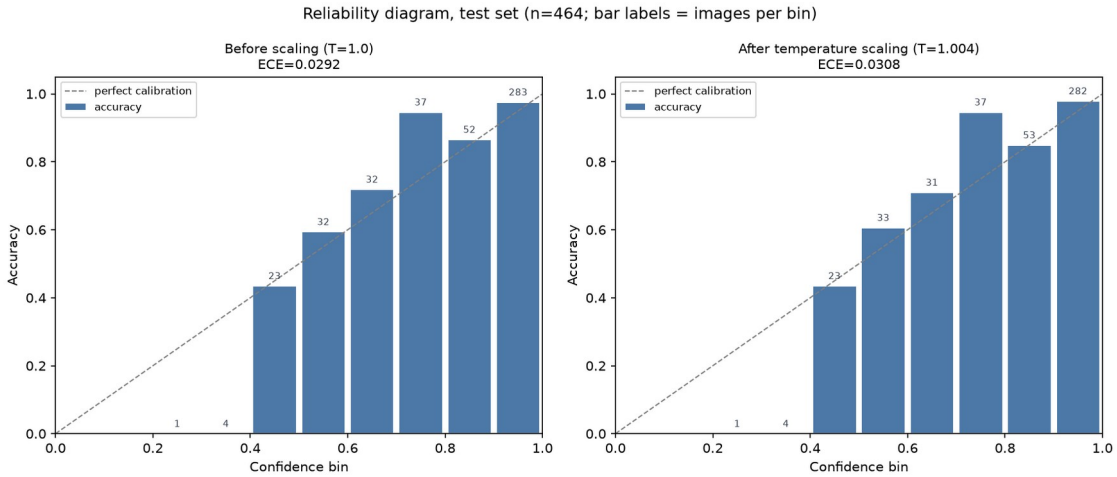


Figure 10. Reliability diagram on the test set, before and after temperature scaling fit on the validation set (analysis only; the deployed demo uses raw softmax confidence).

Expected Calibration Error on the test set was 2.92% with raw softmax (Figure 10, left). Fitting a temperature on the validation set gave T=1.004 (almost no change) and test ECE of 3.08% (Figure 10, right); temperature scaling did not improve calibration here, though the model was already reasonably well calibrated before scaling. The model's accuracy and predicted class are unaffected either way.

6.4 Sample Detection / Recognition Results

56 of the 464 test images (12.1%) were misclassified (464 total minus 408 correct). Paper is recognized most reliably (71 of 75 correct), while Metal is most often confused with Plastic (9 of 119 Metal images). Ten of those misclassified images are shown in Figure 12 (Section 7.3), each captioned with its true class, the predicted class, and the confidence of the wrong prediction. Screenshots of the demonstration program's recognition output are in Appendix D.

6.5 Unseen Situations

An external, cross-dataset test set of 500 images (100 per class) was assembled from TrashNet (Thung & Yang, 2016; MIT License) and evaluated to check generalization beyond RealWaste's own single-facility photography. None of these images were taken by the group. Each image was checked against the RealWaste training images; 0 matches were found, confirming no overlap with the training data. Full source attribution is kept in SOURCES.csv alongside the test images.

- **External test accuracy: 56.00%** (280 of 500), well below the 87.93% RealWaste test accuracy (Section 6.3).

Class	Accuracy (correct / 100)
Cardboard	78% (78/100)
Glass	60% (60/100)
Metal	66% (66/100)
Paper	37% (37/100)
Plastic	39% (39/100)

Table 9. Per-class accuracy on the 500-image (100 per class) unseen TrashNet test set.

Cardboard holds up best on these images at 78%, while Paper falls to 37% and Plastic to 39%.

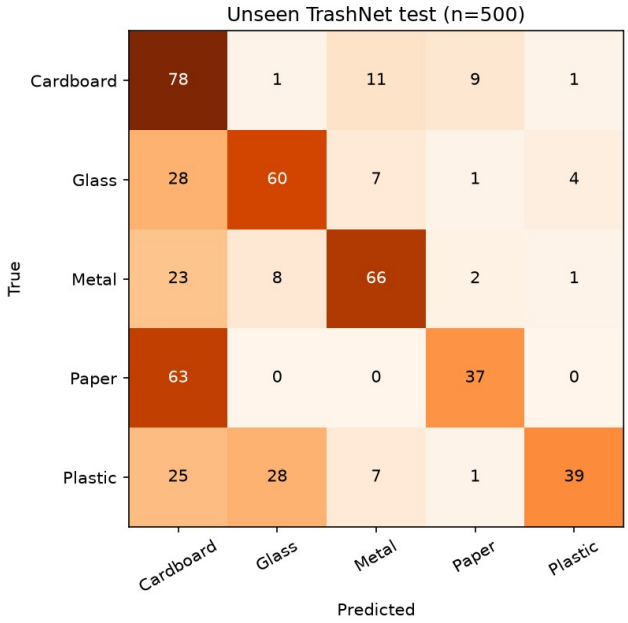


Figure 11. Confusion matrix on the 500-image unseen TrashNet test set.

The drop from 87.93% to 56.00% is domain shift: RealWaste's training photos were all taken at one landfill facility's point of reception, while the external images are everyday, single-object product photos from TrashNet, with different cameras, backgrounds, and lighting the model was never trained on. The largest single error cell is Paper misread as Cardboard (63 of 100 Paper images), and Cardboard is also the most common wrong guess for Glass, Metal, and Plastic images (28, 23, and 25 respectively); Cardboard itself generalizes best (78% correct). Under the per-class confidence thresholds (Table 7, Section 4.6), 16.6% of predictions on this set are flagged Uncertain rather than a wrong label. Full source attribution and license details are kept alongside the test images rather than in this report. This confusion matrix is repeated in Appendix C.

7. RESULTS AND DISCUSSION

7.1 Computer Vision Results

The MobileNetV2 transfer-learning model reaches 87.93% accuracy on the held-out RealWaste test set (5 classes, 464 images), with macro F1 0.878. All five classes score an F1 above 0.86, with no single class dragging the average down.

7.2 Analysis of Results

Paper (recall 94.7%) and Plastic (precision 90.0%) are the most reliably recognized classes. Metal is most often confused with Plastic (9 mistakes out of 464 test images), and Plastic is confused with both Glass (8) and Metal (8), consistent with those classes often containing shiny, reflective, or clear surfaces under the dataset's landfill lighting. Cardboard is most often confused with Paper (7 mistakes), which share similar color and fibrous texture, exactly the confusion the group anticipated in Section 1.4. The 87.93% result is 2.07 percentage points below the 90% objective set in Section 1.3; given that the dataset is imbalanced and visually ambiguous for several class pairs, the group considers this a reasonable result rather than a failed one, with room to close the gap through the recommendations in Section 9.2 rather than a change of approach.

7.3 Errors and Failure Cases

On the RealWaste test set, the dominant failure mode is confusion between visually similar material classes (Metal and Plastic, Plastic and Glass/Metal, Cardboard and Paper; Section 6.3), not random or unexplainable errors. On the external TrashNet test set (Section 6.5), a different and larger confusion pattern appears: Paper misread as Cardboard (63 of 100) is the single biggest error cell, and most other classes' errors also land on Cardboard (Glass 28, Plastic 25, Metal 23), rather than the Metal/Plastic/Glass confusion dominant on the RealWaste test set. This points to the model leaning on texture and color cues that do not reliably separate these classes once the images come from a different camera and setting than RealWaste's own landfill photography, and suggests the model's learned features are biased toward RealWaste's own look for Cardboard in particular.



Figure 12. Ten misclassified test images, captioned true / predicted (confidence).

Figure 12 shows ten misclassified test images, chosen at random, each labeled with the true class, the predicted class, and the model's confidence in that wrong prediction.

8. PROBLEMS ENCOUNTERED AND LIMITATIONS

All training images come from a single landfill facility and camera setup; the external cross-dataset test (Section 6.5) confirms this limits generalization, dropping accuracy from 87.93% to 56.00% on everyday photos from a different source. The selected 5-class subset is imbalanced (Plastic 921 vs. Glass 420 images), addressed with class weights but not eliminated as a source of bias. The system recognizes exactly one of the five trained classes per image; an object outside these classes, or an image containing more than one item, is forced into the closest of the five rather than rejected. A sample of misclassified RealWaste test images is shown in Figure 12 (Section 7.3), but qualitative error analysis beyond that sample still relies mainly on the aggregate confusion matrix (Section 6.3).

9. CONCLUSION AND RECOMMENDATIONS

9.1 Conclusion

The group built and evaluated a 5-class waste-image classifier (Cardboard, Glass, Metal, Paper, Plastic) using MobileNetV2 transfer learning on the RealWaste dataset, satisfying the Midterm requirements for Specialization 8 (Object Sorting): at least 4 classes with at least 100 images each, a prepared train/validation/test split, image preprocessing and augmentation, a working CV model, and evaluation with accuracy metrics and a confusion matrix. The final test accuracy is 87.93% (macro F1 0.878), short of the group's internal 90% target but demonstrating a working SEE and RECOGNIZE pipeline with a visible label and confidence output.

9.2 Recommendations

- **Close the generalization gap found by the external test.** The 87.93%-to-56.00% drop on everyday photos (Section 6.5) is larger than the gap to the 90% RealWaste-test objective; add training images with varied backgrounds and cameras, or stronger augmentation, before relying on the model outside RealWaste's own conditions.
- **Target the Metal/Plastic and Cardboard/Paper confusions.** Consider additional training images or augmentation that emphasizes the cues that distinguish these pairs (e.g., surface reflectance, texture) to close the gap to the 90% objective.
- **Planned for the Final project: robot integration.** The recognized class will feed a decision step that chooses a sorting destination, driving a 4WD chassis's motors through a motor driver to physically move the item (the SEE, DECIDE, and ACT loop), to be demonstrated in the Test 1 (Figure-8 line tracking) and Test 2 (Object Sorting) evaluations. None of this is in scope for this Midterm submission.

REFERENCES

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., … Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems* [Computer software]. https://www.tensorflow.org/

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*.

Chollet, F. (2015). *Keras* [Computer software]. https://keras.io

Chow, C. K. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, *16*(1), 41–46. https://doi.org/10.1109/TIT.1970.1054406

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition* (pp. 248–255). IEEE. https://doi.org/10.1109/CVPR.2009.5206848

Fumera, G., Roli, F., & Giacinto, G. (2000). Reject option with multiple thresholds. *Pattern Recognition*, *33*(12), 2099–2101. https://doi.org/10.1016/S0031-3203(00)00059-5

Geifman, Y., & El-Yaniv, R. (2017). Selective classification for deep neural networks. *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. https://proceedings.neurips.cc/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of the 34th International Conference on Machine Learning*, *70*, 1321–1330. https://proceedings.mlr.press/v70/guo17a.html

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR 2017)*. https://arxiv.org/abs/1610.02136

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kull, M., Perello-Nieto, M., Kängsepp, M., Silva Filho, T., Song, H., & Flach, P. (2019). Beyond temperature scaling: Obtaining well-calibrated multi-class probabilities with Dirichlet calibration. *Advances in Neural Information Processing Systems (NeurIPS 2019)*. https://arxiv.org/abs/1910.12656

Naeini, M. P., Cooper, G., & Hauskrecht, M. (2015). Obtaining well calibrated probabilities using Bayesian binning. *Proceedings of the AAAI Conference on Artificial Intelligence*, *29*(1). https://doi.org/10.1609/aaai.v29i1.9602

Nixon, J., Dusenberry, M. W., Jerfel, G., Nguyen, T., Liu, J., Zhang, L., & Tran, D. (2019). Measuring calibration in deep learning. *arXiv*. https://arxiv.org/abs/1904.01685

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Python Software Foundation. (n.d.). *Python* [Computer software]. https://www.python.org/

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4510–4520). IEEE. https://doi.org/10.1109/CVPR.2018.00474

Single, S., Iranmanesh, S., & Raad, R. (2023). *RealWaste* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5SS4G

Single, S., Iranmanesh, S., & Raad, R. (2023). RealWaste: A novel real-life data set for landfill waste classification using deep learning. *Information, 14*(12), Article 633. https://doi.org/10.3390/info14120633

Thakkar, P. N., Sinha, S., Baijal, K., Bian, Y., Lackey, L., Dodson, B., Kong, H., Kwon, J., Li, A., Hu, Y., Rekoutis, A., Silver, T., & Bhattacharjee, T. (2025). CLAMP: Crowdsourcing a large-scale in-the-wild haptic dataset with an open-source device for multimodal robot perception. *Proceedings of the 9th Conference on Robot Learning*, *305*, 941–960. https://proceedings.mlr.press/v305/thakkar25a.html

Thung, G., & Yang, M. (2016). *TrashNet* [Data set]. GitHub. https://github.com/garythung/trashnet (MIT License)

APPENDICES

Appendix A. Threshold sweep and threshold selection

Section 4.5 explains why the classifier is allowed to answer “Uncertain” instead of forcing a label, and Section 4.6 applies one threshold per predicted class. The two tables below are the validation evidence behind those numbers.

Threshold	Coverage	Accuracy on covered predictions	Wrong predictions caught
0.40	98.28%	86.40%	10.14%
0.50	94.61%	87.70%	21.74%
0.60	88.36%	90.98%	46.38%

Threshold	Coverage	Accuracy on covered predictions	Wrong predictions caught
0.70	81.03%	93.62%	65.22%
0.80	73.49%	95.89%	79.71%
0.90	59.48%	98.55%	94.20%

Table 10. Confidence threshold sweep on the 464-image validation set, the data plotted in Figure 8.

Raising the threshold buys accuracy with coverage. At 0.40 the classifier answers for almost every image but is right on 86.40% of them; at 0.90 it answers for 59.48% and is right on 98.55% of those. The thresholds in Table 7 sit in between, where most images still get an answer.

Measure	Value
Selection rule	smallest t with validation selective accuracy ≥ 90%
Chosen global fallback threshold	0.59
Validation coverage at that threshold	89.01%
Validation accuracy on covered predictions	90.31%
Test coverage at that threshold	87.72%
Test accuracy on covered predictions	93.37%
Test errors caught as Uncertain	29 of 56

Table 11. The global fallback threshold, chosen on the validation set and then applied once to the test set.

The fallback threshold is used only for a class with too few validation predictions to estimate its own threshold reliably (Section 4.5). Applied to the test set it keeps 87.72% of the predictions and is right on 93.37% of them, catching 29 of the 56 test errors instead of reporting them as confident answers.

Appendix B. Confusion matrix on the RealWaste test set

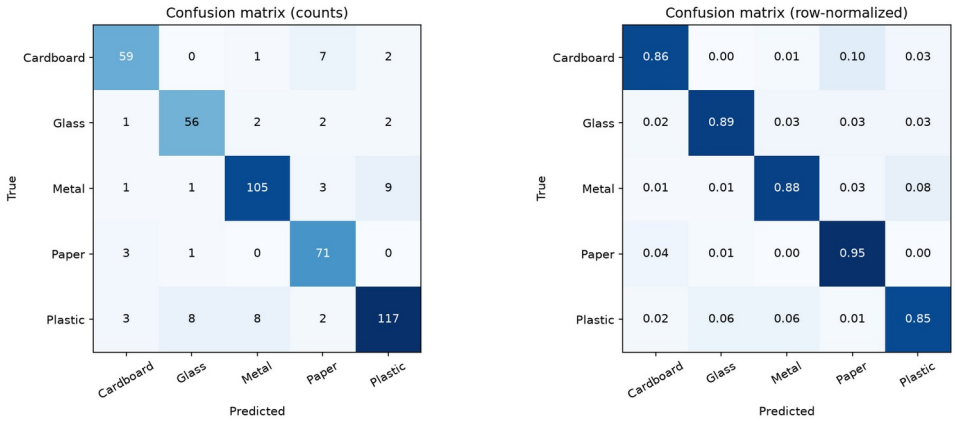


Figure 13. The confusion matrix on the 464-image test set, repeated from Figure 6 (counts, left; row-normalized, right).

The diagonal holds the correct predictions. The off-diagonal cells show that the mistakes cluster on visually similar materials rather than spreading evenly: Metal read as Plastic, Plastic read as Glass or Metal, and Cardboard read as Paper.

Appendix C. Confusion matrix on the unseen TrashNet test set

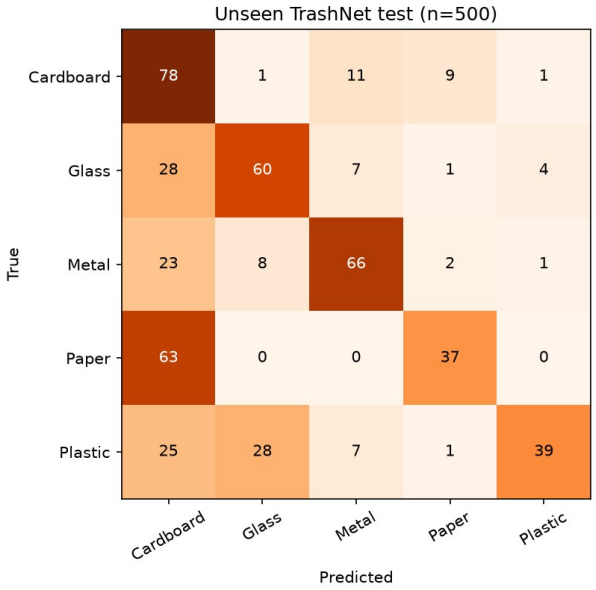


Figure 14. The confusion matrix on the 500-image unseen TrashNet test set, repeated from Figure 11.

The error pattern is different from the one in Appendix B. On images from outside RealWaste the model falls back on Cardboard for every other class, most sharply for Paper, which is the domain shift discussed in Section 6.5.

Appendix D. Recognition output of the demonstration program

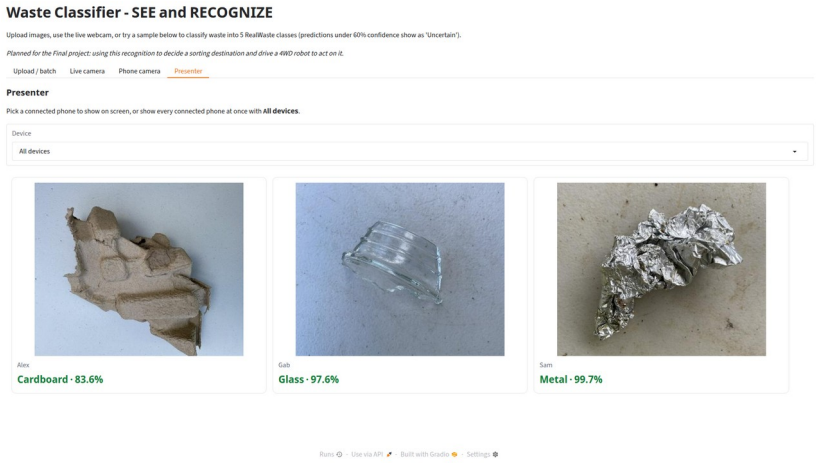


Figure 15. Presenter view of the demonstration program, captured during a user-interface check with three devices connected.

Each card holds one connected device, the image that device sent, and the class and confidence the model returned for it: Cardboard 83.6%, Glass 97.6% and Metal 99.7%. This is a capture of the interface described in Sections 5.2 and 5.3, not a record of the Midterm demonstration itself.



Figure 16. The reject option: a prediction below its threshold is reported as Uncertain together with the best guess.

Rather than forcing a label, the program names the best guess and the confidence it fell short by. Both captures come from an earlier build that compared every prediction against one threshold of 0.60; the program now compares each prediction against that class's own threshold from Table 7.

Appendix E. System architecture diagram

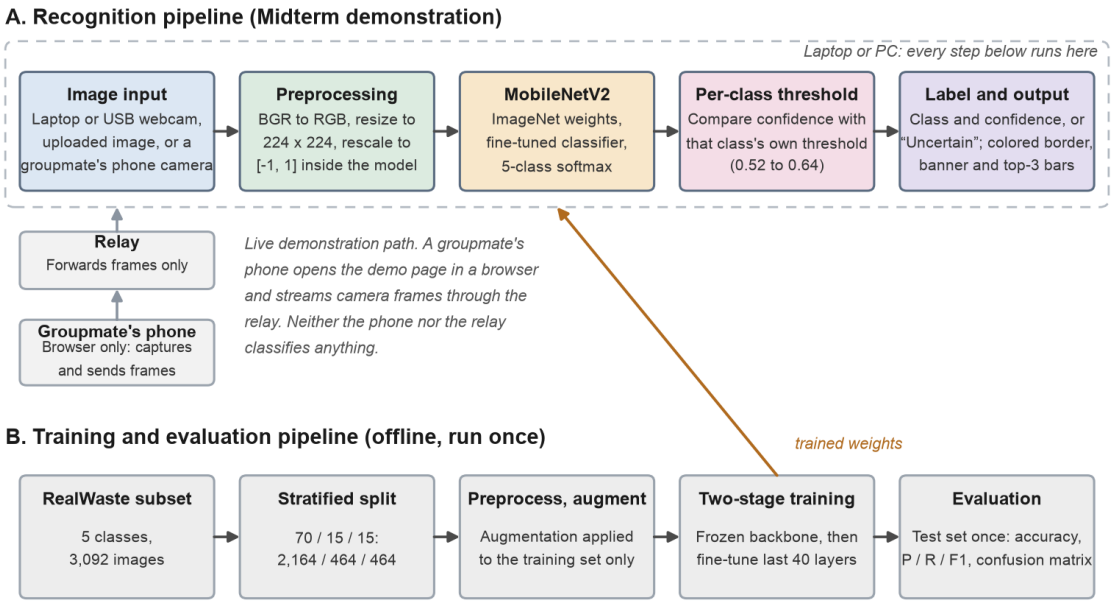


Figure 17. The system architecture, repeated from Figure 1: the recognition pipeline, the live demonstration path, and the offline training pipeline.

Part A is what happens to one image at demonstration time and Part B is the offline run that produced the model it uses. Every step of Part A runs on the processing computer: a connected phone and the relay only move the picture.

Appendix F. Project materials

The notebook, the trained model, the live demonstration program, and the figures and metrics cited in this report are all published in the project repository.

- GitHub repository: https://github.com/J4ve/cv-object-sorting
- Training and evaluation notebook (Google Colab copy): https://colab.research.google.com/drive/1Uunfl1tFrYVt09Iig5-hYnvL3mn288m5?usp=sharing

Inside the repository, the notebook is src/training/Midterm_Object_Sorting_RealWaste.ipynb, the trained model is models/waste_classifier.keras, and the metrics and figures cited here are in results/.